



DIPLOMA EN
CIENCIA DE DATOS

Fundamentos de Bases de Datos y Algoritmos

CLASE #3

Implementación de Bases de Datos

Ilse Gutiérrez Soto

Ingeniera Civil Informática
ILSEGUTIERREZ@UDEC.CL

Aprendizajes del módulo

1. Especificar modelos de datos que permitan describir a alto nivel los requerimientos de persistencia de sistemas de análisis de datos
2. Implementar bases de datos que den respuesta a diversos requerimientos de almacenamiento, análisis y procesamiento de datos

Aprendizajes del módulo

1. Implementar una base de datos relacional por medio de lenguaje declarativo.
2. Usar SQL para consultar y/o manipular datos de una base de datos.

Lenguaje SQL

- Propuesto por IBM en la década de 1970.
- Structured Query Language (SQL):
 - Recuperación,
 - Inserción, actualización y eliminación de datos,
 - Funciones de gestión y administrativas.
- Todos los SGBDs comerciales soportan SQL, pero con extensiones propietarias al lenguaje estándar.

Lenguaje de Definición de Datos (DDL)

- Lenguaje de definición de datos: Asociamos las operaciones de este tipo a la creación, modificación y eliminación de **estructuras** dentro de la base de datos.

Algunas estructuras:

- Tablas
- Columnas
- Claves primarias
- Claves foráneas
- Esquemas
- Índices
- Funciones
- Procedimientos
- Etc

Para este curso ponemos el énfasis en la definición de tablas. Debemos recordar los conceptos de:

- Columnas
- Claves primarias
- Claves foráneas
- Tipos de datos

Lenguaje de Definición de Datos

- Crear
 - **CREATE SCHEMA** <nombre del esquema>;
 - **CREATE TABLE** <nombre de la relación> (<Atributos>;
- Alterar
 - **ALTER TABLE** <nombre de la relación> **ADD** (<Atributos>;
- Borrar
 - **DROP SCHEMA** <nombre del esquema>;
 - **DROP TABLE** <nombre de la relación>;
- Truncar
 - **TRUNCATE TABLE** <nombre de la relación>;

Tipos de Datos para los Dominios

- Cadenas de Caracteres
 - CHAR(*n*) *exactamente n caracteres (longitud fija)*
 - VARCHAR(*n*) *hasta n caracteres (longitud variable)*
- Números
 - INT / INTEGER
 - FLOAT / REAL
- Fechas y horarios (cadenas de caracteres especiales)
 - DATE
 - TIME
- Lógicos - Booleanos
 - BOOLEAN *toma valores TRUE, FALSE, UNKNOWN*

entre muchos más ([Link a documentación](#))

Primary key & Foreign key

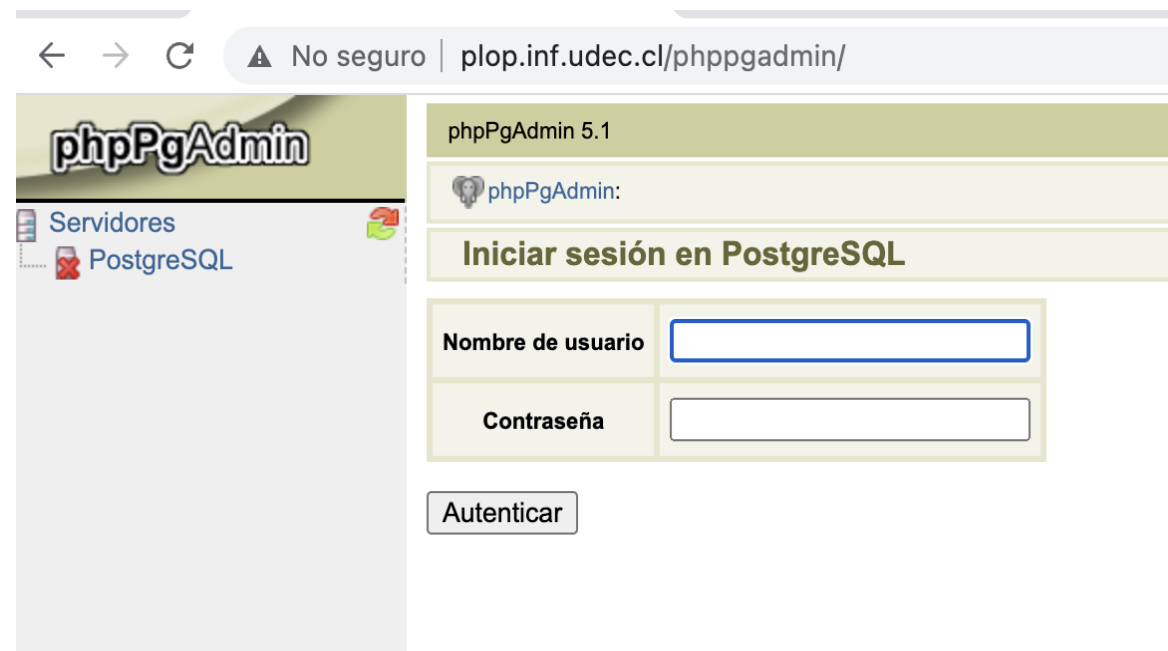
- Primary Key
 - Una llave primaria, o primary key en inglés, es una columna o conjunto de columnas que identifica de manera única cada fila en una tabla de una base de datos relacional.
 - el comando es PRIMARY KEY posterior a la declaración del tipo de dato
- Foreign Key
 - columna o conjunto de columnas en una tabla que hacen referencia a la llave primaria de otra tabla. Esto se utiliza para establecer relaciones entre las tablas y garantizar la integridad referencial de los datos.
 - el comando es REFERENCES(otra_tabla.pk) posterior a la declaración del tipo de dato

PostgreSQL

Para crear la base de datos se requiere de un gestor y para efectos de este curso utilizaremos el SGBD PostgreSQL que veremos a traves de phpPgAdmin.

Ingresaremos en <http://plop.inf.udec.cl/phppgadmin/>

Cada uno tendrá una cuenta para poder probar.



← → ↻ No seguro | plop.inf.udec.cl/phppgadmin/

phpPgAdmin

Servidores
PostgreSQL

phpPgAdmin 5.1

phpPgAdmin:

Iniciar sesión en PostgreSQL

Nombre de usuario

Contraseña

Autenticar

PostgreSQL

← → ↺ No seguro | plop.inf.udec.cl/phppgadmin/ U

phpPgAdmin

PostgreSQL 10.15 (Ubuntu 10.15-0ubuntu0.18.04.1) corriendo en localhost:5432 -- Usted ha iniciado sesión con el usuario "maci_ij_2021q" [SQL](#) | [Historial](#) | [Buscar](#) | [Cerrar sesión](#)

phpPgAdmin: PostgreSQL: maci_ij_2021q:

[Esquemas?](#) [SQL?](#) [Buscar](#) [Variables?](#) [Procesos?](#) [Bloqueos?](#) [Admin](#) [Privilegios?](#) [Exportar](#)

Esquema	Dueño	Acciones			Comentario
☐ public	postgres	Eliminar	Privilegios	Modificar	standard public schema

Acciones múltiples

Seleccionar todos / Seleccionar ninguno ---> --

[Crear Esquema](#)

Servidores

- PostgreSQL
 - maci_ij_2021q
 - Esquemas
 - public
 - Tablas
 - Vistas
 - Secuencias
 - Funciones
 - Búsqueda de
 - Dominios

SQL

Inserte el comando

☒ **Paginar resultados**

Reestablecer

Lenguaje de Definición de Datos

- Consideremos los siguientes esquemas:
 - **IES (Cod_IES, Nombre_IES)**
 - **Carrera(cod_c, nombre_c) PK: cod_c**
 - **Region(id_r, nombre_reg) PK:id_r**
 - **Provincia(id_p, nombre_pro, id_r) PK: id_p; FK: id_r-> Region**
 - **Comuna(id_c,nombre_co, id_p) PK:id_c; FK: id_p-> Provincia**
 - **Sede(id_sede, nombre_sede,cod_IES, id_c) PK: id_sede; FK: id_c-> Comuna, FK: Cod_IES -- IES**
 - **Oferta(id_sede, anno, cod_c, arancel, vacante_s1, vacatne_s2) PK: (id_sede, anno, id_c) FK:id_sede->sede; id_c-> Carrera**

Ejemplos: link para codigo con texto puro

```
CREATE TABLE Carrera (  
  cod_c VARCHAR (50)  
  PRIMARY KEY,  
  nombre_c varchar  
  (300));
```

```
CREATE TABLE Oferta(  
  id_sede INT,  
  anno INT,  
  cod_c VARCHAR(50),  
  Arancel FLOAT,  
  vacante_s1 INT,  
  vacante_s2 INT,  
  PRIMARY KEY (id_sede, anyo, cod_c)  
  FOREIGN KEY (id_sede) REFERENCES Sede (id_sede),  
  FOREIGN KEY (cod_c) REFERENCES Carrera(cod_c));
```

Ejemplos: link para código con texto puro

Supongamos que queremos:

- Agregar un atributo que se olvidó en la tabla Carrera:

ALTER TABLE Carrera **ADD** Area_OECD varchar (40);

- Borrar todos los datos de la tabla carrera:

TRUNCATE TABLE Carrera;

- Borrar la tabla Carrera:

DROP TABLE Carrera;

ACTIVIDAD

- Cree un esquema llamado prueba2
- Cree dentro de este esquema las tablas para carreras, IES y Sede , es decir, Basado en el MER trabajado la clase pasada.

Lenguaje de Manipulación de Datos (DML)

- Lenguaje de manipulación de datos: Asociamos las operaciones de este tipo a la **consulta, creación, modificación y eliminación de datos** dentro de la base de datos.

Lenguaje de Manipulación de Datos

- Insertar
 - **INSERT INTO** <nombre de la relación> (<nombres de atributo>)
VALUES (<valores de atributo>);
- Actualizar
 - **UPDATE INTO** <nombre de la relación>
SET Atributo 1 =valor atributo
WHERE Atributo 2 =valor atributo;
- Borrar
 - **DELETE**
FROM <nombre de la relación>
WHERE Atributo 1 =valor atributo;

Lenguaje de Manipulación de Datos

- Seleccionar – Buscar (Consulta básica)
 - **SELECT** <nombres de atributo>
FROM <nombres de relación>
WHERE <Condición>;

Estructura de consulta básica de SQL

- Seleccionar – Buscar

- **SELECT** <nombres de atributos>

Filtra la columnas que queremos extraer de las tablas

- **FROM** <nombres de las tablas>

Lista la tablas desde donde queremos extraer las columnas

- **WHERE** <Condiciones>;

Filtra las filas que traemos con esta operación en base a una condición simple o compuesta

La podemos leer como:

Seleccionamos las columnas (listado de columnas) **desde** la(s) tabla(s) (listado de tablas) **cuando** se cumpla la condición (inserte condición)

Selecciones y Búsquedas

SELECT [{**ALL|DISTINCT**}] <nombre_campo> [, <nombre_campo>...]
FROM {<nombre_tabla>|<nombre_vista>}[,
{<nombre_tabla>|<nombre_vista>}...]
[**WHERE** <condición> [{**AND|OR**} <condición>...]]
[**GROUP BY** <nombre_campo>[, <nombre_campo>...]]
[**HAVING** <condición> [{**AND|OR**} <condición>...]]
[**ORDER BY** {<nombre_campo>|<indice_campo>} [{**ASC|DESC**}][,
{<nombre_campo>|<indice_campo>} [{**ASC|DESC**}]]];

Selecciones y Búsquedas

WHERE – Comando que restringe los conjuntos a consultar según las condiciones definidas, permite comparaciones y adicionalmente los operadores lógicos AND, OR y NOT.

GROUP BY – Cuando se requiere la información agrupada, se utiliza este comando en conjunto con una función que permita agregar los valores.

HAVING – Condición similar al WHERE, pero que se genera en el subconjunto de datos devueltos de una consulta, se usa en conjunto con GROUP BY.

ORDER BY – comando que nos permite definir en qué orden retornaran de los resultados, siendo posibles de manera ascendente o descendente (**ASC**|**DESC**)

Claúsula WHERE: CONECTORES LÓGICOS

- Las condiciones que podemos aplicar pueden ser simples o compuestas. Para esto usamos los paréntesis en conjunto con los conectores lógicos **AND**, **OR** y **NOT**

Conector lógico	En español
AND	Y
OR	O
NOT	NO

Por ejemplo, buscamos las personas que hayan nacido después de '01-01-1990' y que vivan en Concepción

SELECT nombre, direccion

FROM persona

WHERE fecha_nac > '1990-01-01' **AND** ciudad = 'Concepción';

Claúsula WHERE: BETWEEN

- El operador BETWEEN nos permite simplificar las condiciones que especificamos en la clausula WHERE cuando queremos evaluar un atributo en un rango de valores.
- En términos sencillos nos permite filtrar aquellas filas que se encuentren dentro de determinado rango.
- **Por ejemplo**, buscamos las personas que hayan nacido después de '01-01-1990' y antes del 31-08-2001

SELECT nombre, direccion

FROM persona

WHERE fecha_nac **BETWEEN** '1990-01-01' **AND** '2001-08-2001';

Claúsula WHERE: LIKE

- El operador LIKE nos permite comparar una cadena de texto con un patrón. Con ella podemos determinar cuando un texto empieza, contiene o termina con determinado patrón.
- En términos sencillos nos permite filtrar aquellas filas que coincidan con el patron.

Por ejemplo, buscamos las personas que tengan 'Juan' como parte de su nombre.

```
SELECT nombre, direccion  
FROM persona  
WHERE nombre LIKE '%Juan%';
```

Claúsula WHERE: IN e IN(SELECT)

- El operador IN nos permite especificar múltiples valores para una cláusula WHERE. Podemos usarla en lugar de múltiples condiciones OR. Se puede leer como la palabra “EN” en español.

Por ejemplo, buscamos las personas que viven en alguna ciudad de la provincia de Antofagasta

SELECT nombre, direccion, ciudad

FROM persona

WHERE ciudad **IN** ('Antofagasta', 'Mejillones', 'Sierra Gorda', 'Taltal');

Podemos evaluar si un valor está dentro del resultado de otra consulta SELECT. Esto es lo que llamamos subquery y su aplicación no se limita a este caso.

Lenguaje de Manipulación de Datos: [link](#)

- Consideremos el siguiente esquema que ya fue creado como tabla en la Base de datos:

Carrera

<u>Cod_c</u>	Nombre_c
15	ENFERMERIA
239	INGENIERIA COMERCIAL
4	AUDITORIA

```
CREATE TABLE Carrera (  
cod_c INT PRIMARY KEY,  
nombre_c varchar (50));
```

- ¿Cómo agregamos, modificamos y eliminamos tuplas?

Actividad

- Inserte los datos del ejercicio.

Carrera

<u>Cod_c</u>	Nombre_c
15	ENFERMERIA
239	INGENIERIA COMERCIAL
4	AUDITORIA

Y luego:

- Agregar la carrera 506 Odontología
- Cambiar de nombre a Medicina la carrera 15.
- Eliminar Ingeniería Comercial

Inserción, Modificación y borrado. : [LINK](#)

1. Agregar la carrera 506 Odontología
 - **INSERT INTO** Carrera (cod_c, Nombre_c)
VALUES ('506', 'ODONTOLOGIA');

Carrera

<u>Cod_c</u>	Nombre_c
15	ENFERMERIA
239	INGENIERIA COMERCIAL
4	AUDITORIA

2. Cambiar de nombre a Medicina la carrera 15

UPDATE Carrera

SET nombre_c = 'MEDICINA'

WHERE cod_c = 15;

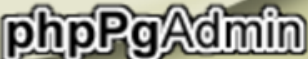
3. Eliminar Ingeniería Comercial

DELETE

FROM Carrera

WHERE nombre_c = 'INGENIERIA COMERCIAL';

PostgreSQL



PostgreSQL 10.15 (Ubuntu 10.15-0ubuntu0.18.04.1) corriendo en localhost:5432 -- Usted ha iniciado sesión con el usuario "maci_ij_2021q"

[SQL](#) | [Historial](#) | [Buscar](#) | [Cerrar sesión](#)

phpPgAdmin: PostgreSQL maci_ij_2021q prueba carrera

Examinar

Acciones		cod_c	nombre_c
Editar	Eliminar	15	ENFERMERIA
Editar	Eliminar	239	INGENIERIA COMERCIAL
Editar	Eliminar	4	AUDITORIA

3 fila(s)

[Atrás](#) | [Expandir](#) | [Insertar](#) | [Refrescar](#)

1. Agregar la carrera 506 Odontología
2. Cambiar de nombre a Medicina la carrera 15
3. Eliminar Ingeniería Comercial

PostgreSQL



PostgreSQL 10.15 (Ubuntu 10.15-0ubuntu0.18.04.1) corriendo en localhost:5432 -- Usted ha iniciado sesión con el usuario "maci_ij_2021q"

[SQL](#) | [Historial](#) | [Buscar](#) | [Cerrar sesión](#)

phpPgAdmin: PostgreSQL?: maci_ij_2021q?: prueba?: carrera?:

Examinar

Acciones		cod_c	nombre_c
Editar	Eliminar	4	AUDITORIA
Editar	Eliminar	506	ODONTOLOGIA
Editar	Eliminar	15	MEDICINA

3 fila(s)

[Atrás](#) | [Expandir](#) | [Insertar](#) | [Refrescar](#)

RESULTADO

Selecciones y Búsquedas

4. ¿Cómo buscamos todas las Universidades que tienen Sede en Concepción?

IES

<u>Cod_IES</u>	Nombre_IES
7	UNIVERSIDAD BOLIVARIANA
19	UNIVERSIDAD DE LAS AMERICAS
54	UNIVERSIDAD TECNOLOGICA DE CHILE INACAP

Sede

<u>Cod_Sede</u>	Nombre_Sede	Comuna_Sede	Cod_IES
13	SEDE CONCEPCION	CONCEPCION	7
3	SEDE LOS ANGELES	LOS ANGELES	7
5	SEDE CONCEPCION	CONCEPCION	19
6	SEDE VIÑA DEL MAR	VIÑA DEL MAR	19
33	SEDE LOS ANGELES	LOS ANGELES	54
8	SEDE VALPARAISO	VALPARAISO	54

```
SELECT nombre_ies
FROM ies
WHERE cod_ies IN (SELECT cod_ies
                  FROM sede
                  WHERE comuna_sede = 'CONCEPCION'));
```

PosgreSQL



PostgreSQL 10.15 (Ubuntu 10.15-0ubuntu0.18.04.1) corriendo en localhost:5432 -- Usted ha iniciado sesión con el usuario "maci_ij_2021q"

[SQL](#) | [Historial](#) | [Buscar](#) | [Cerrar sesión](#)

phpPgAdmin: PostgreSQL? maci_ij_2021q? prueba? ies?

Examinar

Acciones		cod_ies	nombre_ies
Editar	Eliminar	7	UNIVERSIDAD BOLIVARIANA
Editar	Eliminar	19	UNIVERSIDAD DE LAS AMERICAS
Editar	Eliminar	54	UNIVERSIDAD TECNOLOGICA DE CHILE INACAP

3 fila(s)

[Atrás](#) | [Expandir](#) | [Insertar](#) | [Refrescar](#)

PosgreSQL



PostgreSQL 10.15 (Ubuntu 10.15-0ubuntu0.18.04.1) corriendo en localhost:5432 -- Usted ha iniciado sesión con el usuario "maci_ij_2021q"

[SQL](#) | [Historial](#) | [Buscar](#) | [Cerrar sesión](#)

phpPgAdmin: PostgreSQL?: maci_ij_2021q?: prueba?: sede?:

Examinar

Acciones		cod_sede	nombre_sede	comuna_sede	cod_ies
Editar	Eliminar	13	SEDE CONCEPCION	CONCEPCION	7
Editar	Eliminar	3	SEDE LOS ANGELES	LOS ANGELES	7
Editar	Eliminar	5	SEDE CONCEPCION	CONCEPCION	19
Editar	Eliminar	6	SEDE VIÑA DEL MAR	VIÑA DEL MAR	19
Editar	Eliminar	33	SEDE LOS ANGELES	LOS ANGELES	54
Editar	Eliminar	8	SEDE VALPARAISO	VALPARAISO	54

6 fila(s)

[Atrás](#) | [Expandir](#) | [Insertar](#) | [Refrescar](#)

phpPgAdmin: PostgreSQL: maci_ij_2021q:

Resultado de la consulta

nombre_ies
UNIVERSIDAD BOLIVARIANA
UNIVERSIDAD DE LAS AMERICAS

2 fila(s)

Tiempo total de ejecución: 5.430 ms

SQL ejecutada.

[Editar SQL](#) | [Bajar](#)

Servidores

- PostgreSQL
 - maci_ij_2021q
 - Esquemas
 - prueba
 - Tablas
 - carrera
 - ies
 - sede
 - Vistas
 - Secuencias
 - Funciones
 - Búsqueda de f
 - Dominios
 - public

No seguro — plop.inf.udec.cl

SQL?

Buscar

Servidor?: PostgreSQL (localhost:5432:allow)

Base de Datos?: maci_ij_2021q

Orden de búsqueda en los esquemas?:

prueba,public

SELECT nombre_ies

FROM ies, sede

WHERE sede.comuna_sede = 'CONCEPCION' AND ies.cod_ies = sede.cod_ies

☐ Paginar resultados

Ejecutar

Reestablecer

regresar al inicio

ACTIVIDAD

- Consulte todas las carreras impartidas por la Universidad Bolivariana.
- Consulte las carreras que se imparte en Concepción.

Reflexiones

Reflexiones

- ¿Cómo se diferencia SQL de los lenguajes de programación que hemos aprendido?

Consulta SQL v/s Código python equivalente

- Ejemplo consulta SQL
 - **SELECT** nombre, direccion
 - **FROM** persona
 - **WHERE** fecha_nac > '1990-01-01';

```
1  # Suponemos que tenemos una lista de diccionarios, donde cada
2  # diccionario representa a una persona
3  ∨ personas = [
4  ∨      {'nombre': 'Juan',
5          'direccion': 'Calle 123',
6          'fecha_nac': '1985-07-12'},
7  ∨      {'nombre': 'Maria',
8          'direccion': 'Avenida 45',
9          'fecha_nac': '1992-03-29'},
10     # más personas...
11 ]
12
13 # Lista para almacenar el resultado
14 resultado = []
15
16 # Iterar sobre cada persona en la lista
17 ∨ for persona in personas:
18     # Convertir la fecha de nacimiento a una fecha y comparar
19     ∨ if persona['fecha_nac'] > '1990-01-01':
20         # Si la persona nació después del 01-01-1990,
21         # añadir nombre y dirección al resultado
22     ∨     resultado.append({'nombre': persona['nombre'],
23                             'direccion': persona['direccion']})
24
25 # Imprimir el resultado
26 print(resultado)
```

Reflexiones

- ¿Cómo se diferencia SQL de los lenguajes de programación que hemos aprendido?
- Decimos que SQL es un lenguaje **declarativo**:
- No especificamos como extraemos los datos, **nos limitamos a declarar lo que queremos obtener**. No sabemos que operaciones hace internamente el motor de base de datos.
- Con python le decimos explícitamente al computador como proceder. Podemos decir que el lenguaje es **imperativo***

Reflexiones

- ¿Cómo creamos consultas más complejas?
- **Divide y venceras**
 1. Intentamos dividir la consulta en otras consultas más sencillas.
 2. Ejecutamos estas consultas más sencillas.
 3. Las integramos en una única consulta.

